PRACTICAL WORKSHEET 5

CUSTOM DATASET DEVELOPMENT, YOLO11 TRAINING & ROS2 INTEGRATION

MindSpark ROS2 Hands-On Training Module | Ubuntu 24.04 · ROS2 Jazzy · OpenCV · Roboflow · Ultralytics YOLO11

Student / Group	Date	Instructor	Workspace
_____	_____	_____	~/mindspark_ws

Learning Outcomes

- Acquire labelled training material from ROS2 camera recordings.
- Extract frames and annotate the target class using Roboflow.
- Prepare a recording-independent training, validation and test split.
- Fine-tune and evaluate a custom YOLO11 detector using a local NVIDIA GPU.
- Deploy the trained model as a ROS2 executable with an embedded live display.

Practical Overview

Existing /camera_node → /camera/image_raw → existing /video_recorder → MP4 videos → OpenCV frame_extractor.py → Roboflow (green_ball) → YOLO11 dataset → recording-aware splitter → GPU training → best.pt → custom_yolo_detection ROS2 node → live annotated window + topics.

Stage	Evidence from completed MindSpark practical
Video collection	5 recorded MP4 clips
Frame extraction	353 JPEG images extracted (~2 frames/s sampling)
Roboflow dataset	351 exported labelled images; 1 class: green_ball
Corrected split (v2)	Train 166; Valid 144; Test 41; no recording overlap
Model	YOLO11n; 50 epochs; RTX 2060 (6 GB)
Independent test	P 0.998; R 1.000; mAP50 0.995; mAP50–95 0.823
ROS2 integration	ros2 run mindspark_robot custom_yolo_detection; ~7.6 FPS live

Scope note: This worksheet reproduces the final successful workflow. Screenshots of transient setup failures and the unrelated camera colour issue are deliberately excluded. The colour-specific classification weakness observed in live inference is retained as a learning exercise.

Part A — Record the Custom Dataset

A1. Reuse the existing ROS2 camera node

Open Terminal 1; source the ROS2 environment and the workspace. Run the camera node already developed in PW3/PW4.

```
source /opt/ros/jazzy/setup.bash
cd ~/mindspark_ws
source install/setup.bash
ros2 run mindspark_robot camera_node
```

Verify that /camera/image_raw is available and note the actual publisher FPS (observed ~7.68 FPS despite the 30 FPS target).

```
ros2 topic list
ros2 topic hz /camera/image_raw
```

A2. Record five clips using the existing recorder

In Terminal 2, run the recorder from the existing mindspark_robot package. Follow its keyboard interface: R=start, S=stop, Q=quit. Collect multiple examples with varied distance, position and movement; preserve the recording identity in every filename.

source /opt/ros/jazzy/setup.bash

source ~/mindspark_ws/install/setup.bash

ros2 run mindspark_robot video_recorder

ls -lh ~/mindspark_ws/dataset/videos/

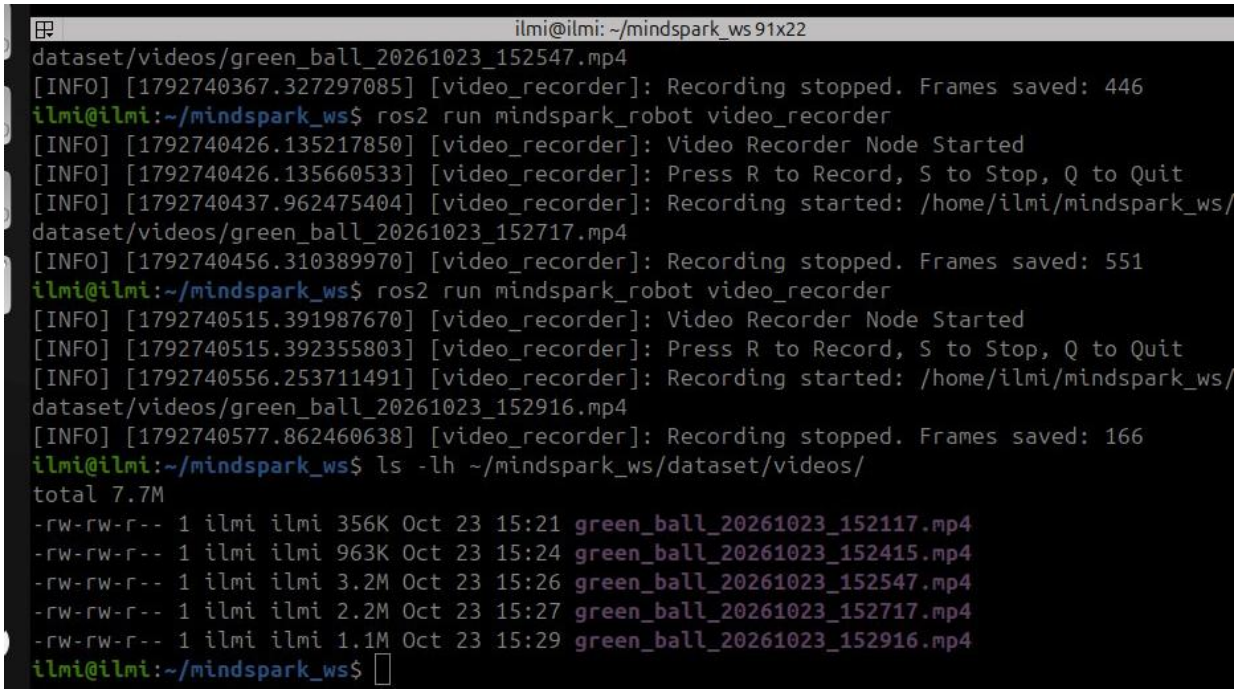


Figure 1. Video recorder output showing saved MP4 clips and frame counts.

Checkpoint A: Record five readable MP4 files in dataset/videos/. Do not confuse encoded video FPS (30) with the actual ROS2 camera publishing FPS (~7.68).

Part B — Extract Video Frames

Video clips are converted to JPEG training examples using a standalone Python script (offline processing; ROS2 is not required). The established extraction rate is approximately 2 frames per second based on the estimated acquisition rate.

B1. Create frame_extractor.py

cd ~/mindspark_ws

source yolo_venv/bin/activate

code frame_extractor.py

Save the script in Appendix A. The output creates one image subdirectory per input recording, retaining the video identifier in each JPEG filename.

B2. Execute and verify

python frame_extractor.py

find ~/mindspark_ws/dataset/images -name "*.jpg" | wc -l

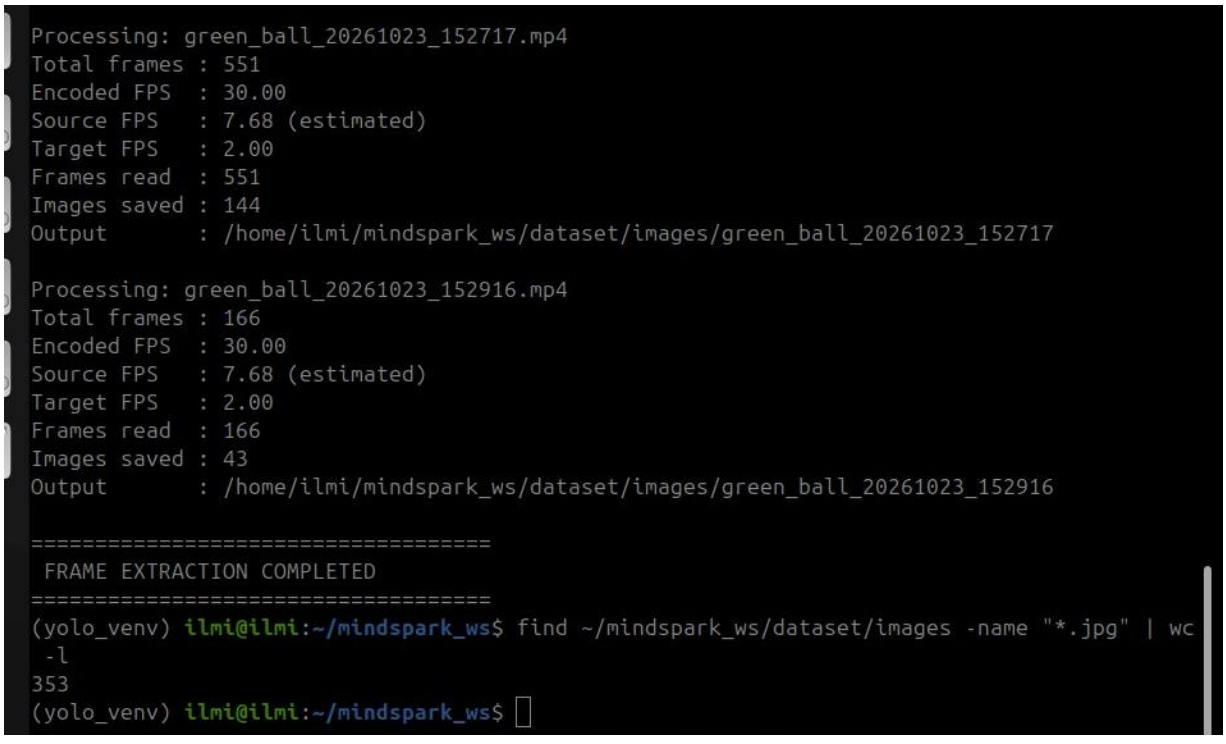


Figure 2. Frame extraction completed: 353 JPEG images produced from the recorded clips.

Checkpoint B: The reference run produced 353 images. Inspect sample frames for blur, annotation feasibility, viewpoint variety and duplicate content.

Part C — Annotate in Roboflow

Create an Object Detection project named MindSpark Green Ball Detection. Upload the extracted JPEG files and define a single class named green_ball. Draw a tight bounding box around each visible target ball.

- Keep bounding boxes consistent. For frames containing no target ball, deliberately retain representative unlabelled/negative examples.
- Do not label blue, yellow or red balls as green_ball; they can later become hard negatives.
- Review annotations before assigning images to the dataset.

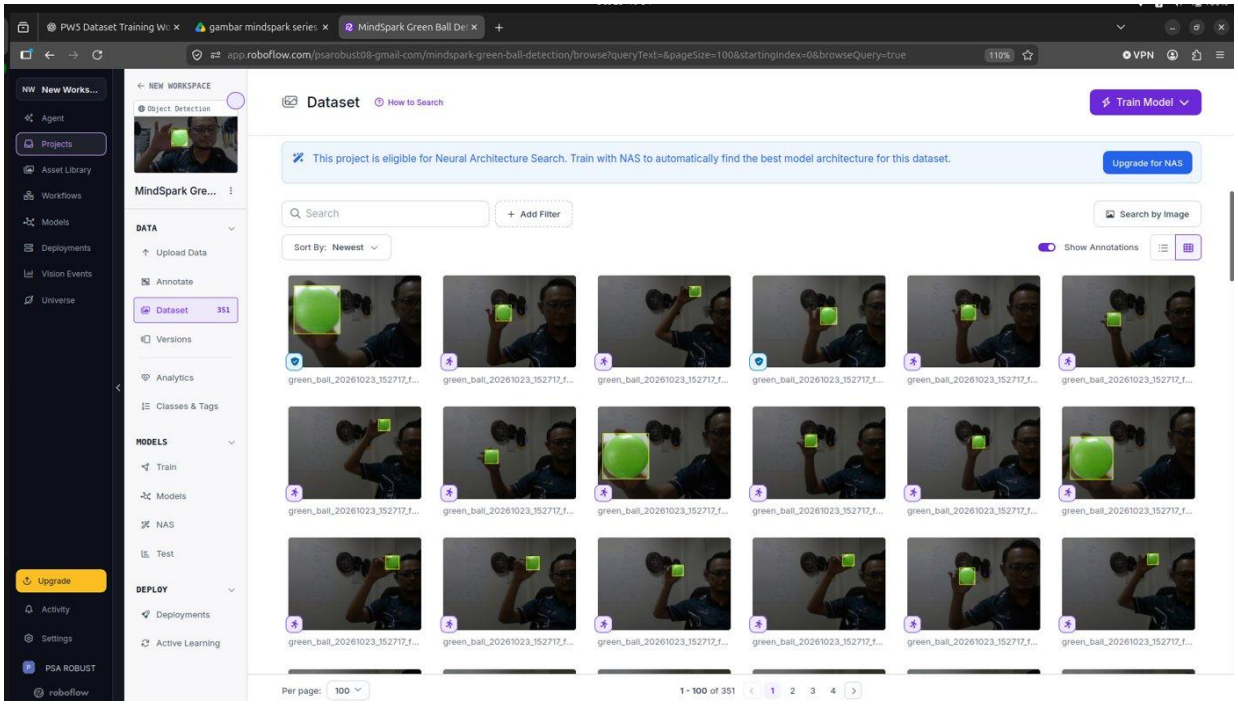


Figure 3. Roboflow annotation view showing labelled green-ball images.

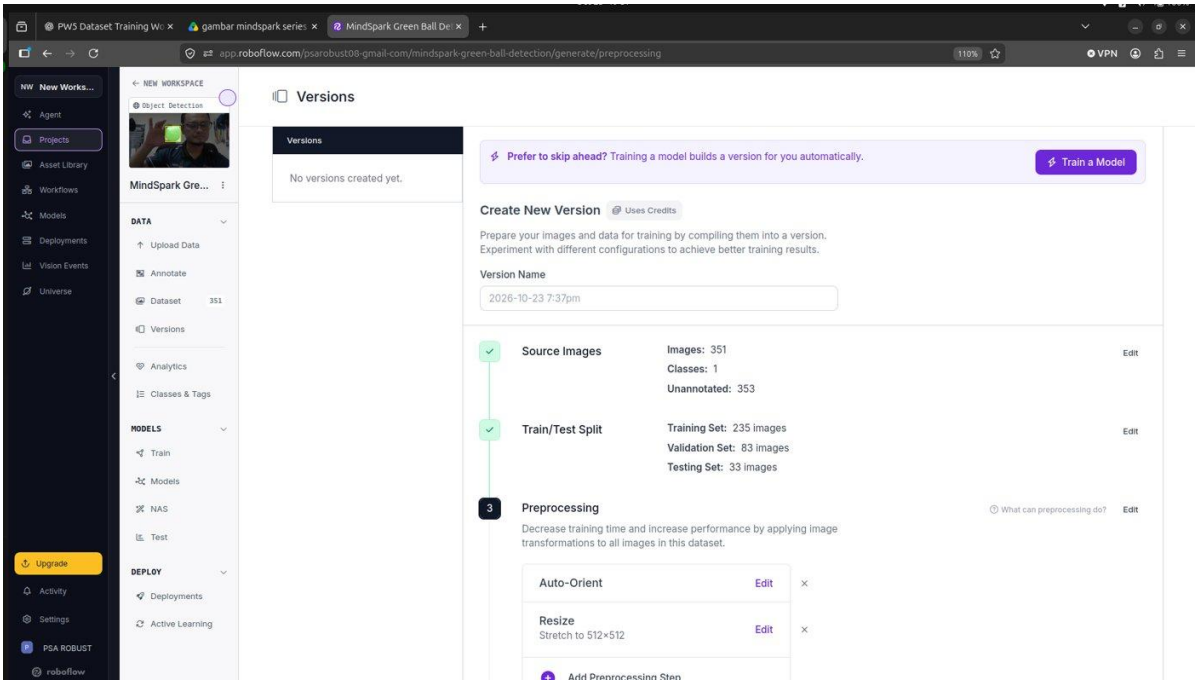


Figure 4. Annotated dataset gallery; bounding-box placement can be reviewed image by image.

C1. Generate and export dataset version

Create Version 1 with Auto-Orient and Resize to 640×640 (Stretch was used in the actual reference run). Do not add extra Roboflow augmentation for this baseline. Export in YOLO11/Ultralytics format as a ZIP for local training.

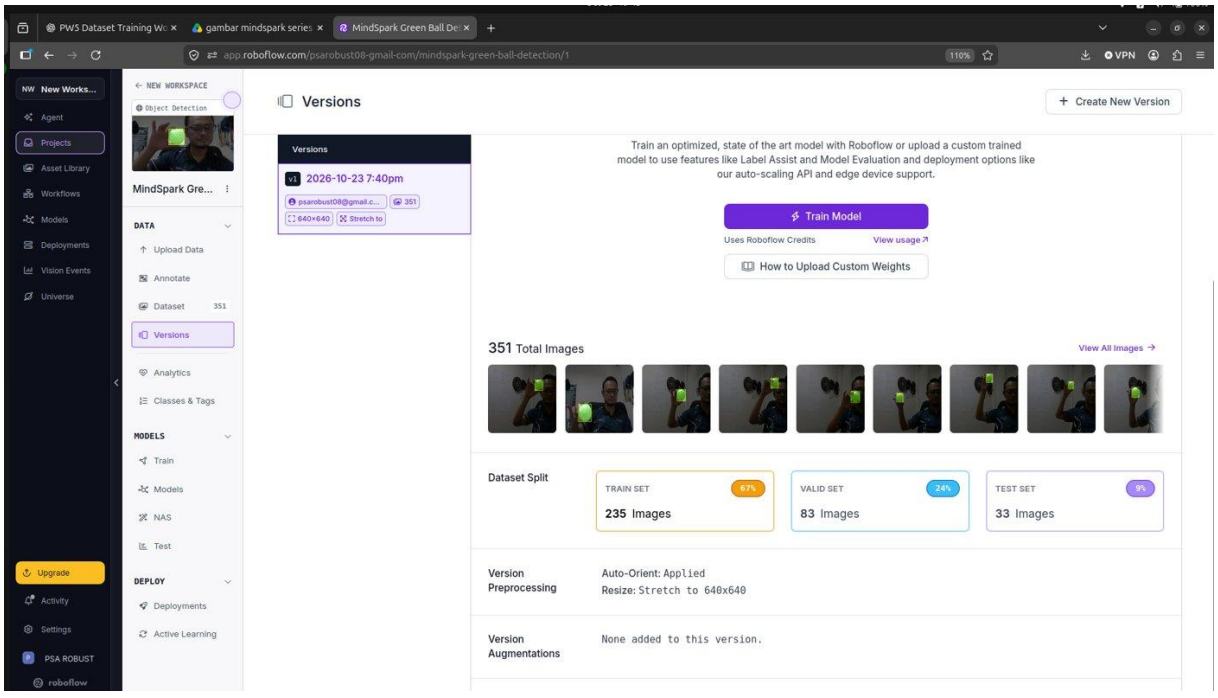


Figure 5. Roboflow Version 1 summary: 351 images; 235/83/33 initial split.

Important: Stretch resizing can distort ball aspect ratio. Prefer aspect-ratio preserving letterbox for later experiments; retain the settings above to reproduce the reference run.

Part D — Prepare an Independent Dataset Split

The initial random Roboflow split put frames from the same video into multiple partitions. This causes correlated examples across train/validation/test and can inflate evaluation metrics.

D1. Copy exported dataset

```
mkdir -p ~/mindspark_ws/dataset/training/green_ball_v1
cp -a ~/Downloads/"MindSpark Green Ball Detection.v1i.yolov11"/.
~/mindspark_ws/dataset/training/green_ball_v1/
```

Check for train/, valid/, test/ and data.yaml, and confirm nc: 1 and names: [green_ball].

D2. Split by original video recording

Create dataset_splitter.py using Appendix B. All frames belonging to one video must go into exactly one split.

Video ID (prefix)	Partition
green_ball_20261023_152117	train
green_ball_20261023_152415	train
green_ball_20261023_152547	train
green_ball_20261023_152717	valid
green_ball_20261023_152916	test

```
cd ~/mindspark_ws
python3 dataset_splitter.py
cat dataset/training/green_ball_v2/data.yaml
```

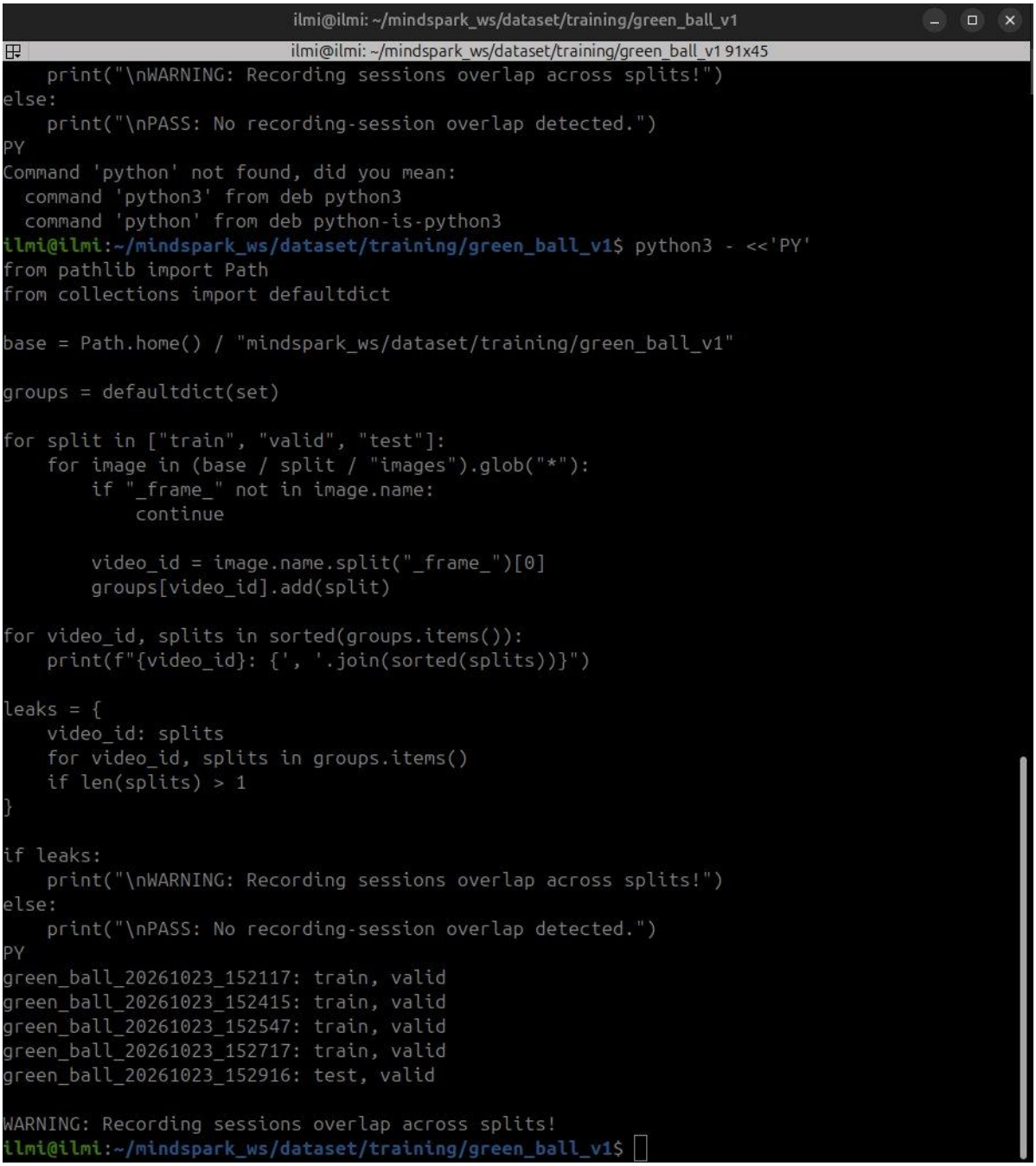


Figure 6. Recording-based splitting completed; corrected distribution is 166/144/41.

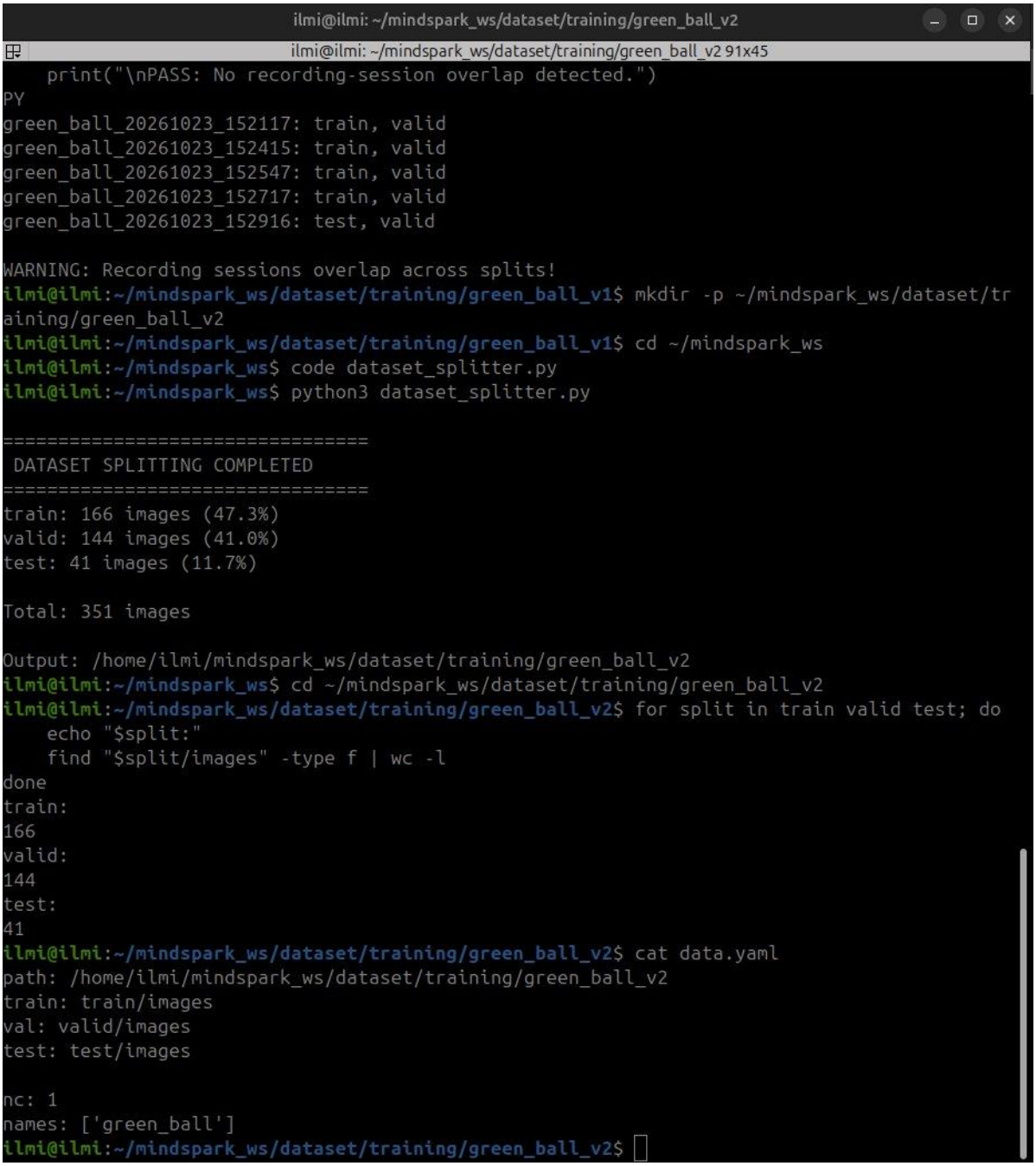


Figure 7. Verification confirms distinct recording sessions in train, validation and test.

Checkpoint D: 351 exported images split into 166 training (47.3%), 144 validation (41.0%) and 41 test (11.7%). Independence by recording takes priority over achieving an exact 70/20/10 ratio. These five recordings still represent a narrow environment.

Part E — Train a Custom YOLO11 Model

Training takes place on the PC rather than Roboflow cloud. Activate the existing venv and verify CUDA before starting.

cd ~/mindspark_ws

source yolo_venv/bin/activate

nvidia-smi

python -c "import torch; print(torch.cuda.is_available()); print(torch.cuda.get_device_name(0))"

```
ilmi@ilmi:~/mindspark_ws/dataset/training/green_ball_v2$ cd ~/mindspark_ws
ilmi@ilmi:~/mindspark_ws$ source yolo_venv/bin/activate
(yolo_venv) ilmi@ilmi:~/mindspark_ws$ which python
/home/ilmi/mindspark_ws/yolo_venv/bin/python
(yolo_venv) ilmi@ilmi:~/mindspark_ws$ nvidia-smi
Fri Oct 23 21:04:44 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.178.04             Driver Version: 580.178.04     CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 2060        Off |   00000000:01:00.0 Off |                  N/A |
| N/A   48C    P8              6W /   80W |       6MiB /   6144MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A            1825      G   /usr/lib/xorg/Xorg                        4MiB |
+-----------------------------------------------------------------------------------------+
(yolo_venv) ilmi@ilmi:~/mindspark_ws$ python -c "import torch; print('PyTorch:', torch.__ve
rsion__); print('CUDA Available:', torch.cuda.is_available()); print('GPU:', torch.cuda.get
_device_name(0) if torch.cuda.is_available() else 'Not available')"
PyTorch: 2.14.1+cu130
CUDA Available: True
GPU: NVIDIA GeForce RTX 2060
(yolo_venv) ilmi@ilmi:~/mindspark_ws$ python -c "import ultralytics; print(ultralytics.__ve
rsion__)"
8.4.183
(yolo_venv) ilmi@ilmi:~/mindspark_ws$
```

Figure 8. PyTorch detects CUDA and the NVIDIA GeForce RTX 2060.

E1. Start fine-tuning

```
yolo detect train \
model=yolo11n.pt \
data=/home/ilmi/mindspark_ws/dataset/training/green_ball_v2/data.yaml \
epochs=50 imgsz=640 batch=8 device=0 workers=4 patience=20 \
project=/home/ilmi/mindspark_ws/dataset/training/results \
name=green_ball_yolo11n_v1
```

For this example, fine-tuning YOLO11n produced model checkpoints best.pt and last.pt. The RTX 2060 was observed using ~3 GB GPU memory during training.

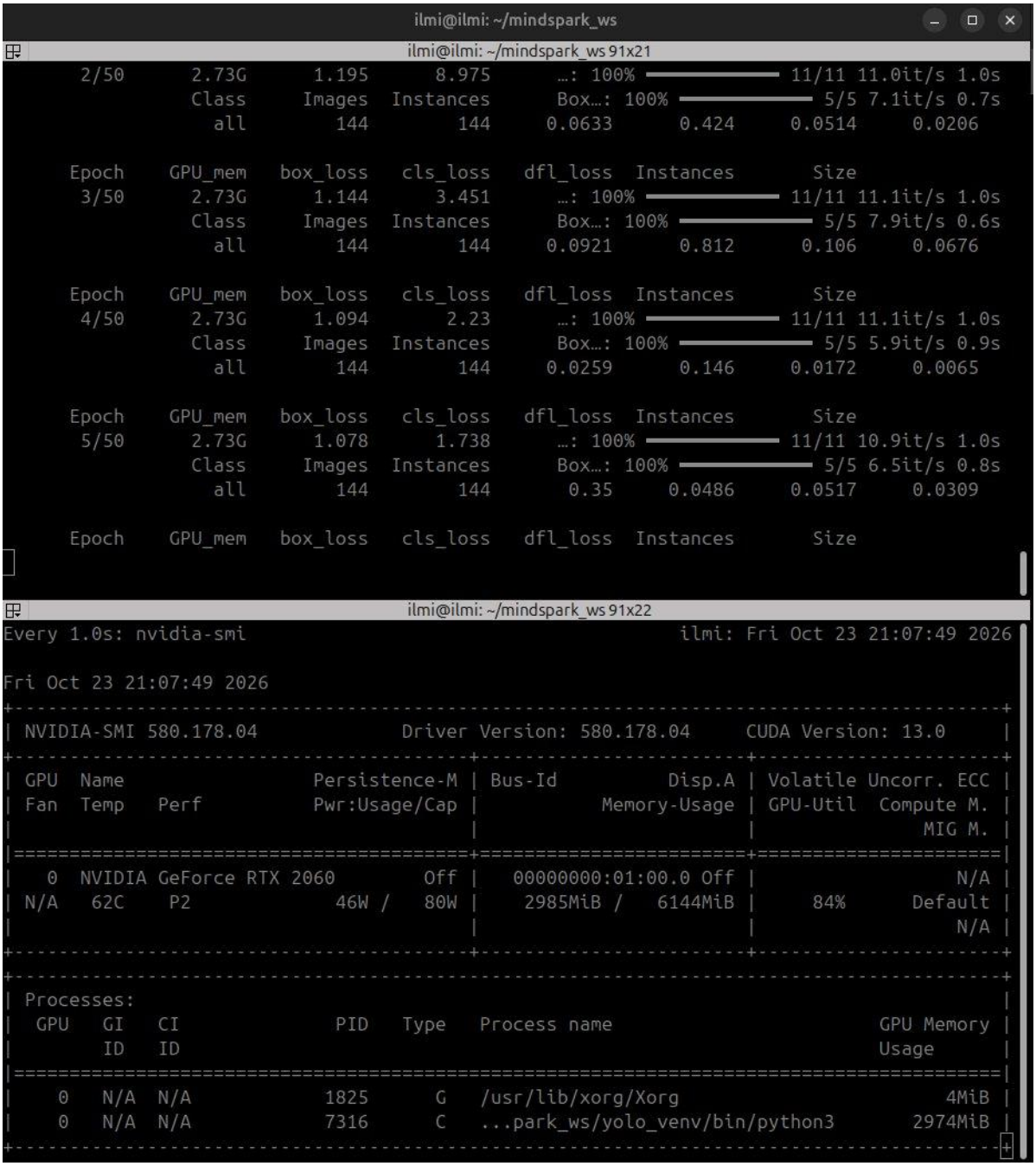


Figure 9. GPU utilisation during YOLO11n training.

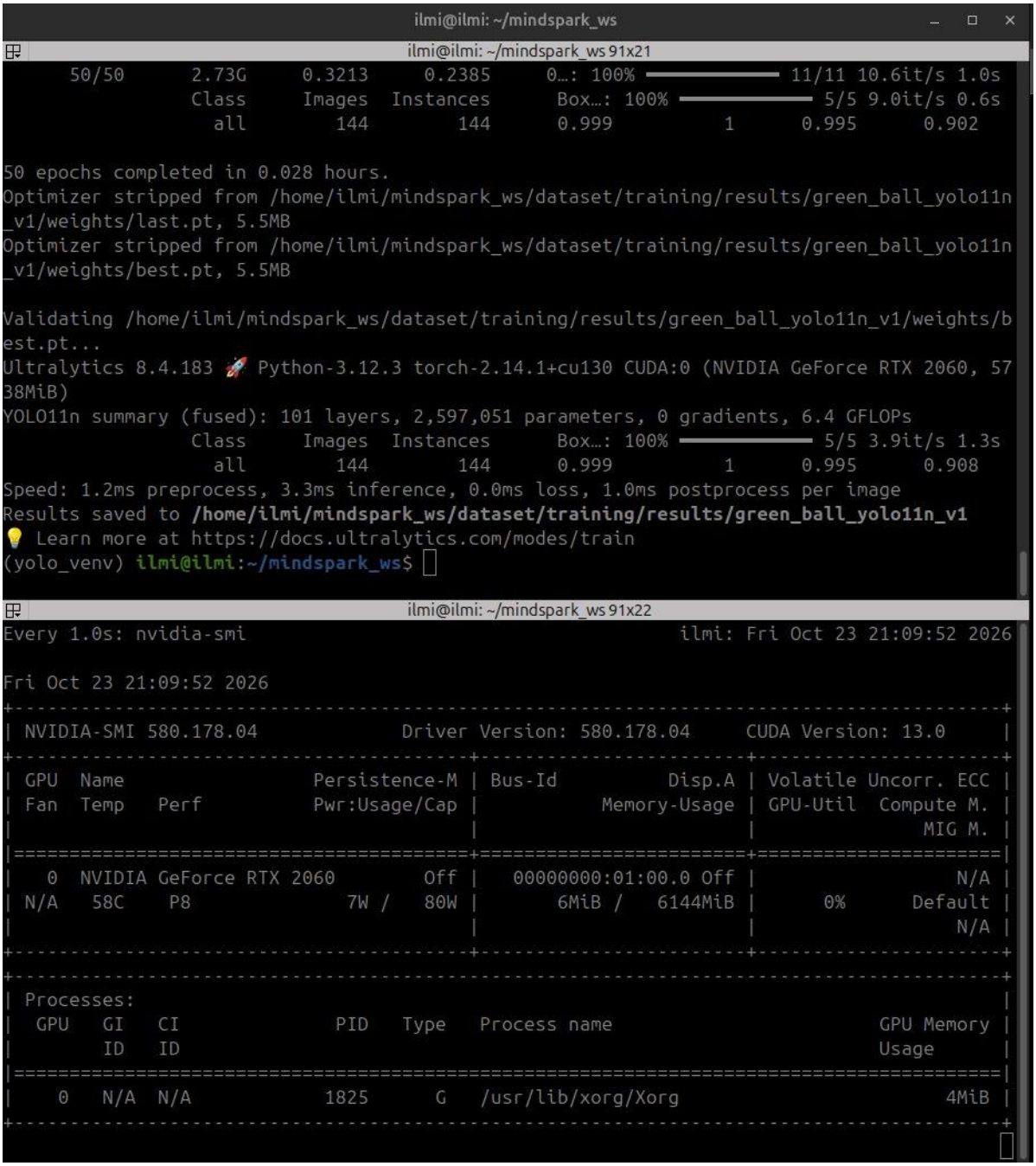


Figure 10. 50 training epochs completed; best.pt generated.

Validation metric	Reported result
Precision	0.999
Recall	1.000
mAP@0.50	0.995
mAP@0.50:0.95	0.908

Checkpoint E: Verify results/green_ball_yolo11n_v1/weights/best.pt exists. Validation scores are not independent test scores.

Part F — Evaluate the Model

Use the test subset only after training and model selection. In the corrected dataset, all 41 test images originate from a held-out recording.

```
yolo detect val \
model=/home/ilmi/mindspark_ws/dataset/training/results/green_ball_yolo11n_v1/weights/best.pt \
data=/home/ilmi/mindspark_ws/dataset/training/green_ball_v2/data.yaml \
split=test imgsz=640 device=0 \
project=/home/ilmi/mindspark_ws/dataset/training/results \
name=green_ball_test
```

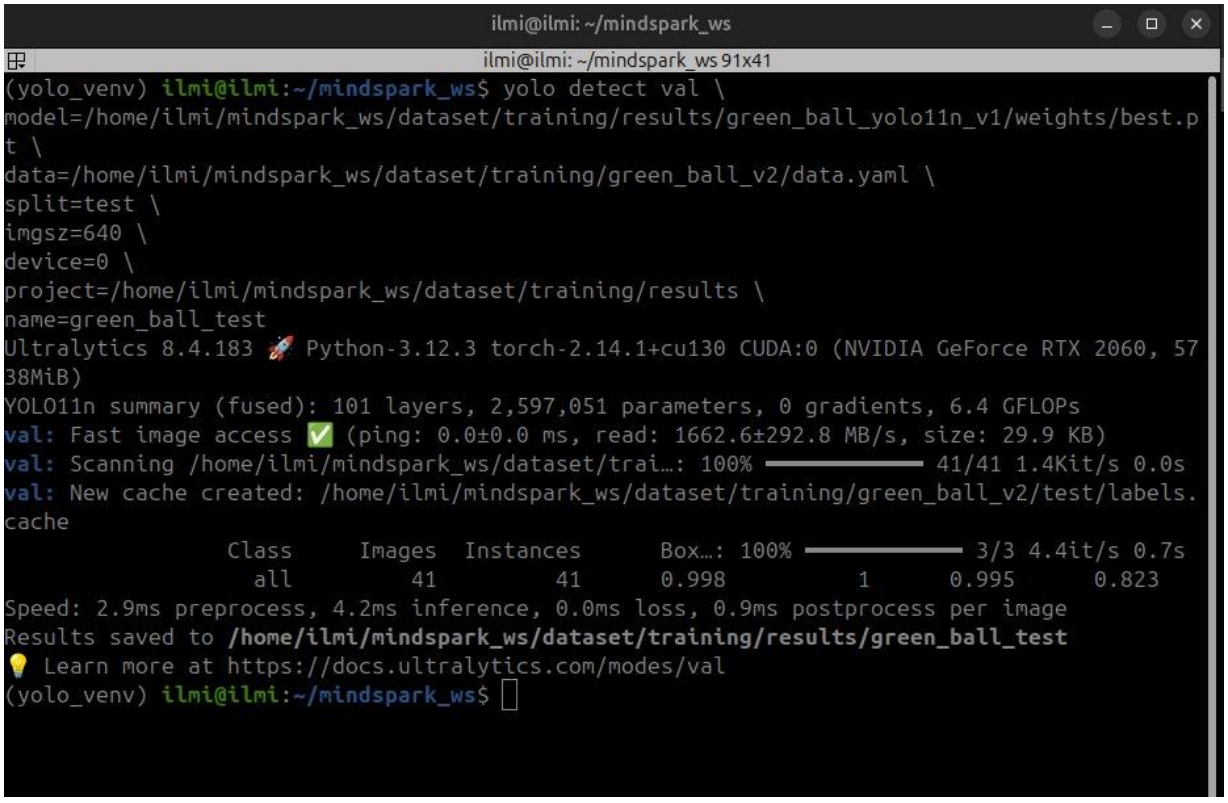


Figure 11. Independent test output from 41 images / 41 annotated ball instances.

Metric	Validation	Test
Precision	0.999	0.998
Recall	1.000	1.000
mAP50	0.995	0.995
mAP50–95	0.908	0.823

The mAP50–95 gap (0.085) suggests less accurate box localisation on the held-out recording. Fast inference reported by Ultralytics is not equal to ROS2 end-to-end camera FPS.

F1. Live webcam challenge

```
yolo detect predict \
model=/home/ilmi/mindspark_ws/dataset/training/results/green_ball_yolo11n_v1/weights/best.pt \
source=0 imgsz=640 conf=0.5 device=0 show=True
```

Present several balls of different colours, with and without the green ball. The baseline model occasionally predicts other ball colours as green_ball; this is a real-world false-positive/generalisation limitation, not proof of colour-specific recognition.

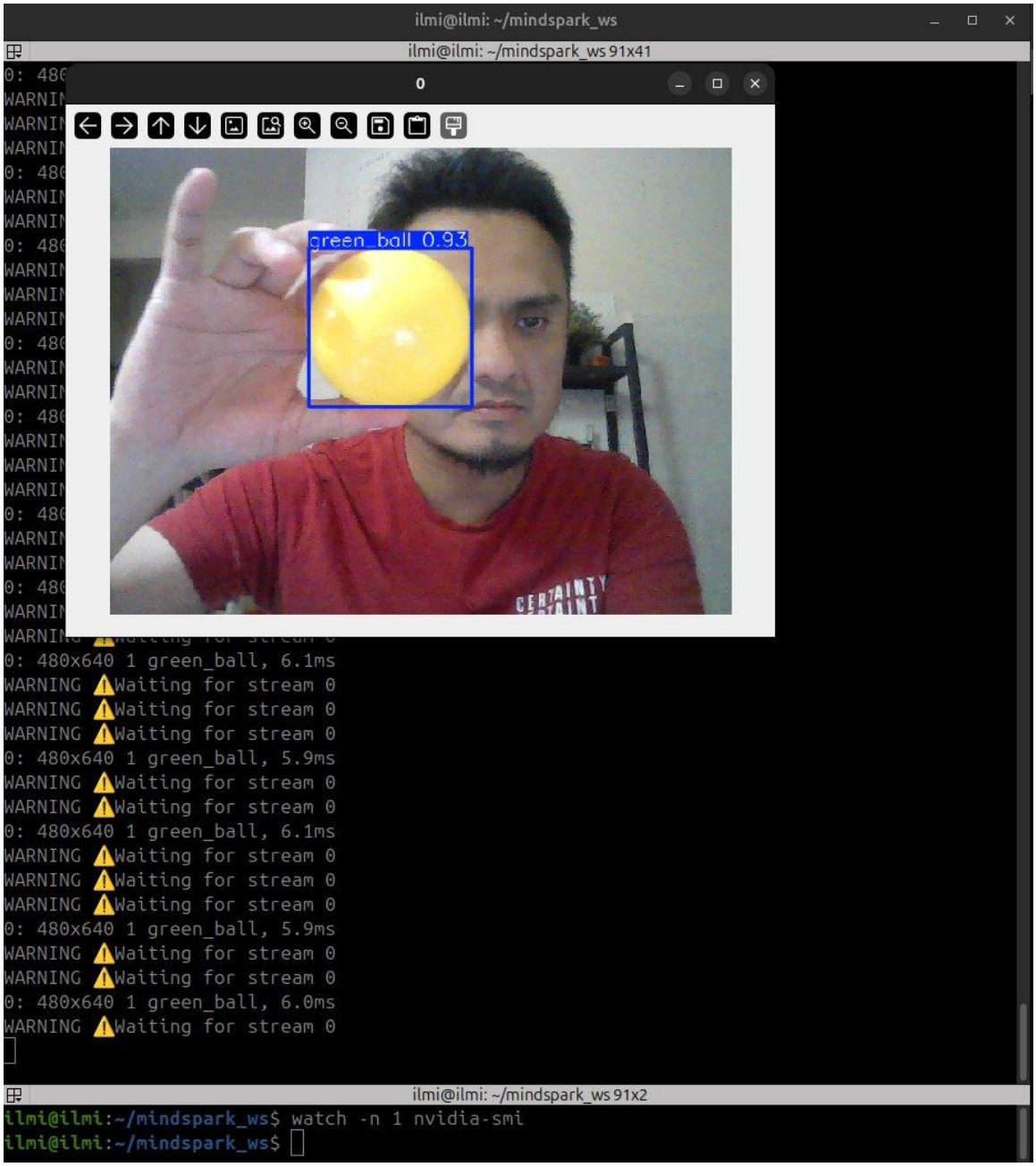


Figure 12. Live challenge demonstrates false-positive detections on non-green balls.

Extension: Collect blue/yellow/red ball hard-negative frames (leave non-green balls unlabelled), include a separate new held-out evaluation session, retrain, and compare false-positive rates.

Part G — Integrate best.pt into ROS2

Reuse the existing camera node. The new custom detector subscribes to /camera/image_raw, runs YOLO11 on the GPU, publishes an annotated image and the highest-confidence detected centre, and opens an OpenCV live window directly.

Interface	Topic	Type
Input	/camera/image_raw	sensor_msgs/msg/Image
Output 1	/custom_yolo/annotated_image	sensor_msgs/msg/Image
Output 2	/custom_yolo/ball_position	geometry_msgs/msg/Point

Position convention used in this practical: Point.x and Point.y are pixel coordinates; Point.z temporarily stores detector confidence (not depth). If no ball is detected, no new position message is published, so downstream clients must handle stale data.

G1. Node and entry point

Save the final functional source as mindspark_robot/custom_yolo_detection_node.py (Appendix C). In setup.py, add the following console script while preserving all previous entries:

'custom_yolo_detection = mindspark_robot.custom_yolo_detection_node:main',

For annotated-image viewer compatibility, use RELIABLE QoS (queue depth 10) for the annotated output publisher. The input camera subscription retains qos_profile_sensor_data.

G2. Build using the YOLO virtual environment

cd ~/mindspark_ws

source /opt/ros/jazzy/setup.bash

source yolo_venv/bin/activate

python -m colcon build --packages-select mindspark_robot --symlink-install

source install/setup.bash

This project required python -m colcon to generate a console script with the venv interpreter, rather than /usr/bin/python3.

head -n 1 install/mindspark_robot/lib/mindspark_robot/custom_yolo_detection

Expected: #!/home/ilmi/mindspark_ws/yolo_venv/bin/python

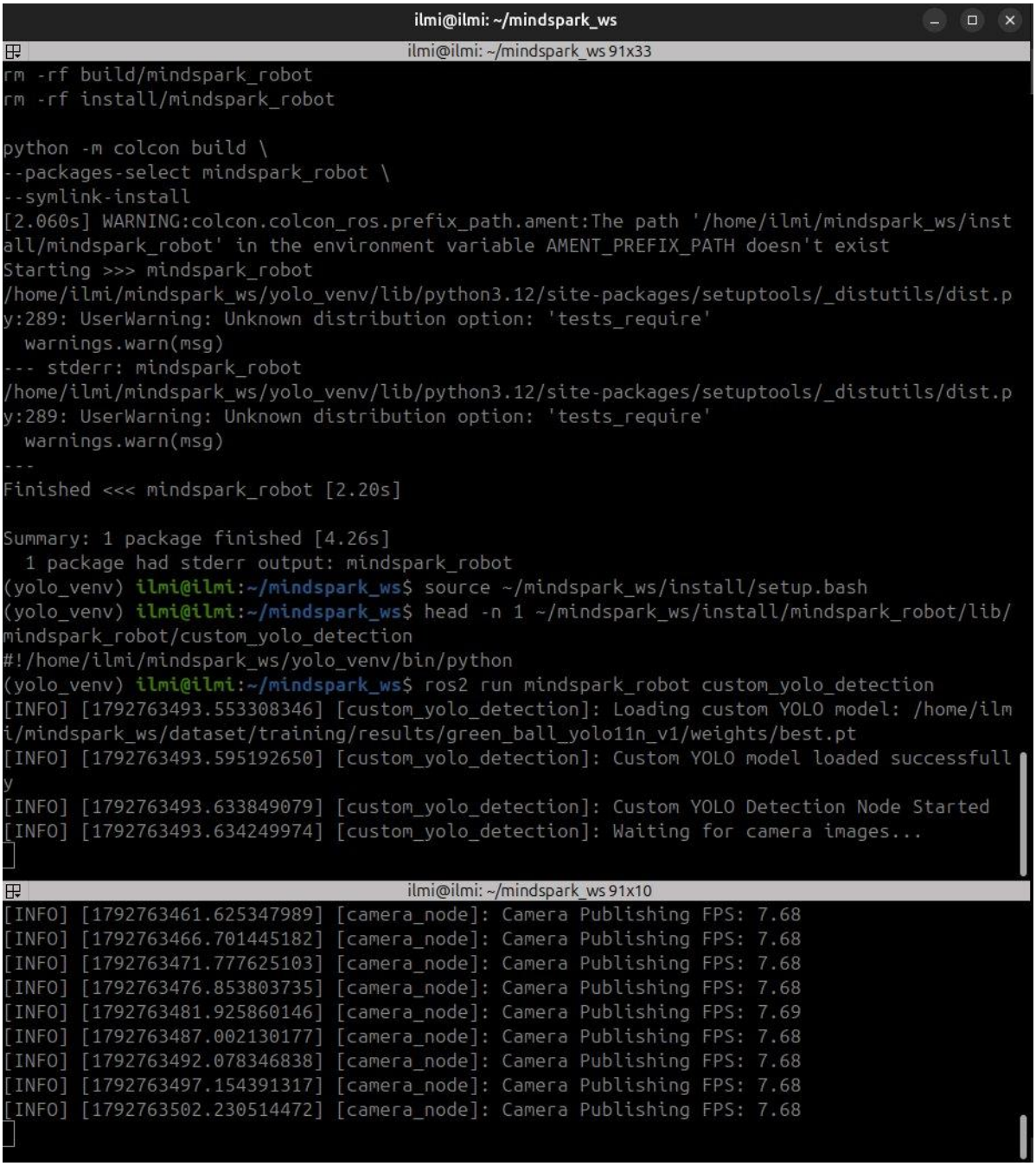


Figure 13. Verified ROS2 entry point built with the yolo_venv Python interpreter.

G3. Run the live ROS2 detection workflow

Terminal 1: Start the camera node. Terminal 2: activate yolo_venv and run the custom executable. An OpenCV window opens automatically; no separate image_tools showimage command is necessary for normal classroom use.

```
ros2 run mindspark_robot camera_node
# in a separate terminal (venv activated):
ros2 run mindspark_robot custom_yolo_detection
```

Verify the outputs if required using ros2 topic list -t, ros2 topic hz /custom_yolo/annotated_image and ros2 topic echo /custom_yolo/ball_position. Press Q in the live detection window to exit.

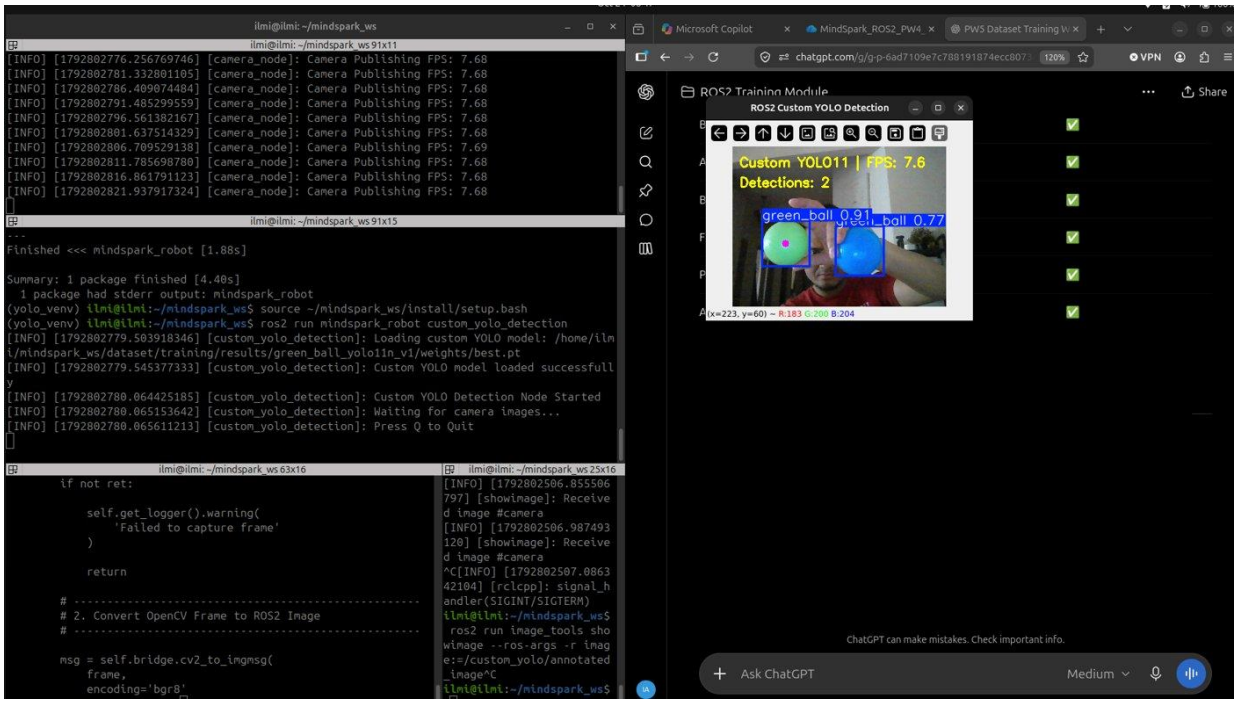


Figure 14. Final successful ROS2 Custom YOLO Detection window (~7.6 FPS) launched directly from ros2 run.

Checkpoint G: The custom model loads, multiple bounding boxes are drawn, the annotated window is visible, and /custom_yolo/annotated_image is actively published. The model still needs hard-negative improvement to distinguish colour robustly.

Student Recording Sheet

Checkpoint	Student result / evidence
Number of MP4 recordings	_____
Number of extracted JPG images	_____
Annotation class name	_____
Train / validation / test totals	_____ / _____ / _____
CUDA GPU model	_____
Number of training epochs	_____
Test Precision / Recall	_____ / _____
Test mAP50 / mAP50–95	_____ / _____
Live ROS2 annotated FPS	_____
Incorrect colour detections observed?	Yes / No; notes: _____

Discussion Questions

- Why is the 30 FPS encoded-video value different from the 7.68 FPS camera publishing measurement?
- How can frames from the same recording across both train and test sets bias the evaluation?
- Why can a high mAP50 coexist with false positives on similarly shaped but differently coloured balls?
- Why did ros2 run initially fail when the executable used /usr/bin/python3 rather than the YOLO virtual environment?
- What is the difference between a camera topic, an annotated-image topic and a ball-position topic?

Appendix A — frame_extractor.py (reference implementation)

```
#!/usr/bin/env python3
import cv2
from pathlib import Path
BASE = Path.home() / "mindspark_ws" / "dataset"
VIDEO_DIR, IMAGE_DIR = BASE / "videos", BASE / "images"
SOURCE_FPS, TARGET_FPS, IMAGE_QUALITY = 7.68, 2.0, 95

def extract_frames(video_path):
    output = IMAGE_DIR / video_path.stem
    output.mkdir(parents=True, exist_ok=True)
    cap = cv2.VideoCapture(str(video_path))
    if not cap.isOpened():
        print("Cannot open:", video_path)
        return
    frame_index, saved = 0, 0
    next_sample_time = 0.0
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        timestamp = frame_index / SOURCE_FPS
        if timestamp >= next_sample_time:
            target = output / f"{video_path.stem}_frame_{frame_index:06d}.jpg"
            if cv2.imwrite(str(target), frame,
                           [cv2.IMWRITE_JPEG_QUALITY, IMAGE_QUALITY]):
                saved += 1
            next_sample_time += 1.0 / TARGET_FPS
        frame_index += 1
    cap.release()
    print(video_path.name, "read:", frame_index, "saved:", saved)

if __name__ == "__main__":
    IMAGE_DIR.mkdir(parents=True, exist_ok=True)
    for f in sorted(VIDEO_DIR.glob("*.mp4")):
        extract_frames(f)
```

Timing limitation: SOURCE_FPS is an estimated average publishing FPS; exact elapsed-time sampling would require reliable recording timestamps. The present method suffices for instructional frame sampling.

Appendix B — dataset_splitter.py (recording-aware export)

```
#!/usr/bin/env python3
from pathlib import Path
from collections import defaultdict
import shutil
BASE = Path.home() / "mindspark_ws/dataset/training"
SOURCE, DEST = BASE / "green_ball_v1", BASE / "green_ball_v2"
ASSIGN = {
 "green_ball_20261023_152117": "train",
 "green_ball_20261023_152415": "train",
 "green_ball_20261023_152547": "train",
 "green_ball_20261023_152717": "valid",
 "green_ball_20261023_152916": "test",
}
if DEST.exists() and any(DEST.iterdir()):
    raise SystemExit("Output v2 is not empty; review it before rerunning.")
counts = defaultdict(int)
for source_split in ("train", "valid", "test"):
    for image in (SOURCE/source_split/"images").iterdir():
        if image.suffix.lower() not in (".jpg", ".jpeg", ".png"):
            continue
        prefix = image.name.split("_frame_")[0]
        if prefix not in ASSIGN:
            raise ValueError(f"Unrecognised recording: {image.name}")
        target_split = ASSIGN[prefix]
        target_image = DEST/target_split/"images"
        target_label = DEST/target_split/"labels"
        target_image.mkdir(parents=True, exist_ok=True)
        target_label.mkdir(parents=True, exist_ok=True)
        shutil.copy2(image, target_image/image.name)
        label = SOURCE/source_split/"labels"/(image.stem+".txt")
        if label.exists():
            shutil.copy2(label, target_label/label.name)
        counts[target_split] += 1
(DEST/"data.yaml").write_text(
 f"path: {DEST}\ntrain: train/images\nval: valid/images\n"
 "test: test/images\nnc: 1\nnames: ['green_ball']\n"
)
for k in ("train", "valid", "test"):
```

```
    print(k, counts[k])
print("Total:", sum(counts.values()))
```

Appendix C — custom_yolo_detection_node.py (final working version)

```
#!/usr/bin/env python3
import os
import time
import cv2
import rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from sensor_msgs.msg import Image
from geometry_msgs.msg import Point
from cv_bridge import CvBridge
from ultralytics import YOLO

class CustomYOLODetectionNode(Node):
    def __init__(self):
        super().__init__('custom_yolo_detection')
        default_model = os.path.expanduser(
            '~/mindspark_ws/dataset/training/results/'
            'green_ball_yolo11n_v1/weights/best.pt')
        self.declare_parameter('model_path', default_model)
        self.declare_parameter('confidence', 0.5)
        self.declare_parameter('device', '0')
        self.model_path = self.get_parameter('model_path').value
        self.confidence = self.get_parameter('confidence').value
        self.device = self.get_parameter('device').value
        if not os.path.isfile(self.model_path):
            raise FileNotFoundError(self.model_path)
        self.model = YOLO(self.model_path)
        self.bridge = CvBridge()
        self.subscription = self.create_subscription(
            Image, '/camera/image_raw', self.image_callback,
            qos_profile_sensor_data)
        self.image_publisher = self.create_publisher(
            Image, '/custom_yolo/annotated_image', 10)
        self.position_publisher = self.create_publisher(
            Point, '/custom_yolo/ball_position', 10)
        self.window_name = 'ROS2 Custom YOLO Detection'
```

```
        cv2.namedWindow(self.window_name, cv2.WINDOW_NORMAL)
        self.quit_requested = False
        self.previous_time = time.perf_counter()
        self.fps = 0.0
        self.get_logger().info('Custom YOLO Detection Node Started')
        self.get_logger().info('Waiting for camera images...')
        self.get_logger().info('Press Q to Quit')

    def image_callback(self, msg):
        try:
            frame = self.bridge.imgmsg_to_cv2(
                msg, desired_encoding='bgr8')
        except Exception as err:
            self.get_logger().error(f'Image conversion error: {err}')
            return
        results = self.model.predict(
            source=frame, conf=self.confidence,
            device=self.device, verbose=False)
        result = results[0]
        annotated_frame = result.plot()
        detections = []
        for box in result.boxes:
            class_id = int(box.cls[0].item())
            if self.model.names[class_id] != 'green_ball':
                continue
            confidence = float(box.conf[0].item())
            x1, y1, x2, y2 = box.xyxy[0].tolist()
            detections.append((confidence, (x1+x2)/2, (y1+y2)/2))
        if detections:
            confidence, cx, cy = max(detections, key=lambda d: d[0])
            point = Point()
            point.x, point.y, point.z = cx, cy, confidence
            self.position_publisher.publish(point)
            cv2.circle(annotated_frame, (int(cx), int(cy)),
                       5, (255, 0, 255), -1)
        now = time.perf_counter()
        dt = now - self.previous_time
        if dt > 0:
            self.fps = 1.0/dt
```

```
        self.previous_time = now
        cv2.putText(annotated_frame,
                    f'Custom YOLO11 | FPS: {self.fps:.1f}', (10,30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0,255,255), 2)
        cv2.putText(annotated_frame,
                    f'Detections: {len(detections)}', (10,60),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0,255,255), 2)
        image_msg = self.bridge.cv2_to_imgmsg(
            annotated_frame, encoding='bgr8')
        image_msg.header = msg.header
        self.image_publisher.publish(image_msg)
        cv2.imshow(self.window_name, annotated_frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            self.quit_requested = True

def main(args=None):
    rclpy.init(args=args)
    node = None
    try:
        node = CustomYOLODetectionNode()
        while rclpy.ok() and not node.quit_requested:
            rclpy.spin_once(node, timeout_sec=0.1)
    except KeyboardInterrupt:
        pass
    finally:
        if node is not None:
            node.destroy_node()
        cv2.destroyAllWindows()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Implementation note: OpenCV imshow() requires a local desktop GUI; this teaching-oriented node is not intended as a headless or fully optimised deployment implementation.